

Rand Protocol: A Permissionless Privacy Layer for Solana

Post-Quantum Confidential Transfers via a Shielded Pool
on the Solana Virtual Machine

Dendi Suhubdy
Bitwyre Research
dendi@bitwyre.com

December 21, 2025

Abstract

We present Rand, a privacy layer for Solana. Rather than launching a new Layer 1, Rand deploys as a set of Solana programs implementing a multi-asset shielded pool, together with an off-chain relayer network. Consensus, ordering, and settlement are inherited from Solana’s Proof of History and its validator set; Rand introduces no consensus mechanism of its own and is not a validator client. This choice is deliberate: it lets confidential transfers operate directly over the native USDC and USDT mints already circulating on Solana, with no bridging and no wrapped assets, and it inherits Solana’s throughput and fee structure rather than attempting to reproduce them.

Confidentiality is achieved with a Zcash-style note and nullifier model. Shielded notes are commitments in a concurrent Merkle tree; spends publish nullifiers and a zero-knowledge proof establishing membership, correct nullifier derivation, spend authority, and value conservation. We adopt a deliberate asymmetry in our post-quantum posture, which we argue is the correct engineering tradeoff rather than a compromise: note encryption (ML-KEM), commitments (Poseidon), and in-circuit spend authorization (ML-DSA) are post-quantum, because ciphertexts published to a public ledger are permanently exposed to harvest-now-decrypt-later attacks. Proof soundness uses Groth16 over BN254, verified on-chain through Solana’s `alt_bn128` syscalls within a single 1232-byte transaction. Soundness is a live-network property enforced by an upgradeable program, so it can migrate to a post-quantum argument system when on-chain verification of one becomes economical; confidentiality cannot be retrofitted after the fact.

A single native token, SHRUGG, pays for confidential transfers. We prove that a user who funds a shielded operation with transparent SOL thereby deanonymizes it, which makes an in-circuit fee token a structural requirement rather than an economic addition: SHRUGG is spent from within the shielded pool to compensate relayers who submit transactions and pay SOL fees on the user’s behalf.

Proofs are generated by the user by default, and may optionally be delegated to a permissionless market of GPU provers who race for batches drawn from a public queue, the first submission included by Solana consuming the batch’s nullifiers and collecting its fees. We state the cost of that option rather than presenting it as free: a prover holding a witness learns the sender, recipient, and amount regardless of what the resulting argument conceals from third parties, so delegation exchanges confidentiality against one chosen party for confidentiality against everyone else.

Keywords: Solana, Shielded Pool, Zero-Knowledge Proofs, Confidential Transfers, Post-Quantum Cryptography, Groth16, Relayers, Proving Market, Solana Virtual Machine

Contents

1	Introduction	4
1.1	Contributions	4
1.2	Paper Organization	5
2	Preliminaries	5
2.1	Notation	5
2.2	Cryptographic Hash Functions	5
2.3	Digital Signatures	5
2.4	Commitment Schemes	5
2.5	Zero-Knowledge Proofs	6
2.6	Merkle Accumulators	6
3	System Architecture	7
3.1	Overview	7
3.2	Token Model	7
3.3	Rationale for a Shielded Fee Token	8
3.4	Relayer Economics	8
3.5	Delegated Proving and the Proving Market	9
3.6	Resource Accounting	10
3.7	Participant Roles	10
3.8	Solana Virtual Machine Integration	10
4	Settlement on Solana	11
4.1	Inherited Properties	11
4.2	Consequences of Inheritance	11
4.3	Concurrency	11
5	Privacy-Preserving Transactions	12
5.1	Private State Model	12
5.2	Private Transaction Circuit	12
5.3	Proof System and On-Chain Verification	13
5.4	Nullifier Set	13
5.5	Note Discovery	13
5.6	Encrypted Outputs	14
6	Supply Auditability and Transparency	14
6.1	Motivation for Supply Auditability	14
6.2	SHRUGG Supply Auditability	15
6.3	Privacy-Preserving Supply Verification	15
7	Zero-Knowledge Proof Generation Pipeline	15
7.1	Overview	15
7.2	Step 1: Rust Circuit Definition	15
7.3	Step 2: Rank-1 Constraint System	16
7.4	Performance Characteristics	17
7.5	End-to-End Proving Timeline	17
8	Token Economics	18
8.1	Initial Token Distribution	18
8.2	Emission	18
8.3	Fee Distribution	18

9	Security Analysis	19
9.1	Inherited Consensus Security	19
9.2	Privacy Security	19
9.3	Post-Quantum Security	19
10	Implementation Considerations	20
10.1	Development in Rust	20
10.2	Performance Targets	21
11	Related Work	21
12	Conclusion	22

1 Introduction

Solana provides high throughput, sub-second block times, and the deepest stablecoin liquidity of any high-performance chain: native USDC and USDT circulate directly on it, issued by Circle and Tether rather than bridged. What it does not provide is transaction privacy. Balances, counterparties, and amounts are public by default, which rules out a large class of legitimate uses—payroll, treasury operations, supplier payments, trading strategy—for any participant unwilling to publish their entire financial history.

Existing approaches each give up something essential. Privacy-focused Layer 1s such as Zcash [6] and Monero [8] provide strong anonymity but exist as separate networks with their own assets, their own liquidity, and their own security budgets; a user wanting private USDC cannot use them without bridging. Solana’s own Token-2022 confidential transfer extension [15] encrypts amounts using twisted ElGamal but leaves sender and recipient in plaintext, providing confidentiality without an anonymity set; moreover, because the extension must be enabled at mint creation, it cannot be applied to the existing native USDC and USDT mints, which are legacy SPL Token accounts. Privacy-preserving rollups obtain their own fee token and execution environment at the cost of bridged assets and fragmented liquidity.

Rand takes the opposite approach. We do not build a chain. We observe that a shielded pool is expressible as a Solana program, that Solana already provides the pairing arithmetic needed to verify a succinct proof on-chain [14], and that the assets worth transacting privately are already present. Rand is therefore a privacy layer deployed *onto* Solana: it inherits consensus, ordering, finality, and liquidity, and adds only what Solana lacks.

We note explicitly what this rules out. Rand is not a Solana validator client in the sense of Agave or Firedancer. Those are interchangeable implementations of identical consensus rules, and a client that accepted transactions its peers rejected would fork the network rather than extend it. New transaction semantics on Solana come from programs, and that is where Rand lives.

1.1 Contributions

Our main contributions are:

- A multi-asset shielded pool construction for Solana operating directly over native USDC, USDT, and SOL, with a single shared anonymity set across assets
- A proof that transparent fee payment deanonymizes shielded operations, establishing an in-circuit fee token as a structural requirement, and a relayer mechanism satisfying it
- A permissionless proving market in which unbonded provers race for batches, together with a theorem establishing that delegated proving discloses the witness and is therefore a confidentiality boundary rather than a performance optimisation
- A post-quantum threat analysis separating confidentiality from soundness, with justification for post-quantum note encryption alongside classically-sound proofs
- A circuit specification and on-chain verification design fitting Solana’s 1232-byte transaction and compute-unit limits
- Concurrency treatment for shielded state under Solana’s parallel execution, including recent-root windows and concurrent Merkle tree updates
- Supply auditability for SHRUGG under partial shielding

1.2 Paper Organization

Section 2 provides cryptographic preliminaries. Section 3 describes the system architecture and the shielded fee token argument. Section 4 explains what Rand inherits from Solana. Section 5 details the shielded transfer construction. Section 6 explains supply auditability. Section 7 covers the proof generation pipeline. Section 8 analyzes token economics. Section 9 provides security analysis. Section 10 discusses implementation. Section 11 reviews related work, and Section 12 concludes.

2 Preliminaries

2.1 Notation

Let λ denote the security parameter. We write $x \leftarrow S$ for uniform sampling from set S and $x \leftarrow \mathcal{A}(\cdot)$ for the output of algorithm $\mathcal{A}$. We use $\{0,1\}^*$ for arbitrary-length bit strings and $\{0,1\}^\lambda$ for λ -bit strings. $\epsilon(\lambda)$ denotes a negligible function.

2.2 Cryptographic Hash Functions

Definition 2.1 (Collision-Resistant Hash Function). *A hash function family $\mathcal{H} = \{H_\lambda : \{0,1\}^* \rightarrow \{0,1\}^\lambda\}_{\lambda \in \mathbb{N}}$ is collision-resistant if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$:*

$$\Pr[(x, x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge H_\lambda(x) = H_\lambda(x')] \leq \epsilon(\lambda) \quad (1)$$

We employ SHA-3 (Keccak-256) [2] and BLAKE3 [9] as our primary hash functions, both providing 128-bit post-quantum security against Grover’s algorithm [5].

2.3 Digital Signatures

Definition 2.2 (Digital Signature Scheme). *A digital signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ consists of:*

- $\text{KeyGen}(1^\lambda) \rightarrow (pk, sk)$: *Key generation*
- $\text{Sign}(sk, m) \rightarrow \sigma$: *Signing algorithm*
- $\text{Verify}(pk, m, \sigma) \rightarrow \{0,1\}$: *Verification algorithm*

satisfying correctness and existential unforgeability under chosen message attacks (EUF-CMA).

For post-quantum security, we employ CRYSTALS-Dilithium [4], a lattice-based signature scheme standardized by NIST. Dilithium signatures provide:

- Public key size: 1,312 bytes
- Signature size: 2,420 bytes
- Security level: NIST Level 2 (128-bit quantum security)

2.4 Commitment Schemes

Definition 2.3 (Commitment Scheme). *A commitment scheme $\text{Comm} = (\text{Setup}, \text{Commit}, \text{Open})$ satisfies:*

- **Hiding:** *For all PPT adversaries, $\text{Comm}(m_0; r_0)$ is computationally indistinguishable from $\text{Comm}(m_1; r_1)$*

- **Binding:** No PPT adversary can find (m, r) and (m', r') with $m \neq m'$ such that $\text{Comm}(m; r) = \text{Comm}(m'; r')$

Rand commits to notes with the Poseidon hash function [11]:

$$\text{Comm}(m; r) = \text{Poseidon}(m \| r) \quad (2)$$

We prefer this to Pedersen commitments, despite the latter’s additive homomorphism, for two reasons. Poseidon is algebraic over the proving field and costs roughly two orders of magnitude fewer constraints than a bitwise hash such as SHA-256, which dominates circuit size in practice. More importantly, Pedersen hiding rests on the discrete logarithm assumption and is therefore broken by a quantum adversary, whereas Poseidon’s hiding is preserved. Since commitments are published permanently, this distinction matters for the reasons developed in Section 9.3.

2.5 Zero-Knowledge Proofs

Definition 2.4 (Zero-Knowledge Argument System). *A zero-knowledge argument system for relation R consists of $(\text{Prove}, \text{Verify})$ satisfying:*

- **Completeness:** For $(x, w) \in R$, $\Pr[\text{Verify}(x, \text{Prove}(x, w)) = 1] = 1$
- **Soundness:** For $x \notin L_R$ and PPT $\mathcal{A}$: $\Pr[\text{Verify}(x, \mathcal{A}(x)) = 1] \leq \epsilon(\lambda)$
- **Zero-Knowledge:** $\exists$ PPT simulator $\mathcal{S}$ such that $\{\text{Prove}(x, w)\}_{(x, w) \in R} \approx_c \{\mathcal{S}(x)\}_{x \in L_R}$

We employ Groth16 [10] over the BN254 curve as our argument system. This choice is driven by a hard platform constraint rather than by preference, and we state the reasoning explicitly because it departs from the transparent, post-quantum proof systems otherwise attractive for this application.

A Solana transaction is limited to 1232 bytes, derived from the IPv6 minimum MTU. A Groth16 proof is three curve points, approximately 256 bytes, and therefore fits alongside its public inputs in a single transaction. Verification consists of a small fixed number of pairings, available as the `alt_bn128` syscalls [14], and completes well inside the per-transaction compute budget.

STARKs [1] are transparent and post-quantum, and were the natural candidate. They are nonetheless impractical here: a FRI-based proof for this circuit is 50–200 KB, so submitting one would require first uploading it in roughly 1 KB fragments across on the order of 200 transactions, paying rent on the accumulating account, and only then verifying it across further transactions. The resulting cost and latency per transfer are not defensible for a payment system. We revisit this tradeoff, and the conditions under which it reverses, in Section 9.3.

Groth16 requires a per-circuit trusted setup. Because Rand’s circuit is fixed rather than general-purpose, this is a one-time cost, discharged by a public multi-party ceremony in which security holds if any single participant is honest.

2.6 Merkle Accumulators

Definition 2.5 (Merkle Accumulator). *A Merkle accumulator over a set S is a binary tree whose leaves are the elements of S and whose internal nodes are hashes of their children. The root rt commits to S ; membership of $x \in S$ is proven by a path of $O(\log |S|)$ sibling hashes.*

Rand uses accumulators for two distinct purposes with different requirements. Note commitments are held in an append-only tree whose root must be updatable by many concurrent writers within a single Solana slot; we use a concurrent Merkle tree with a changelog buffer for this. Nullifiers require efficient *non-membership* decisions, which we address in Section 5.

Because Rand operates as a program on Solana rather than as an independent network, Byzantine agreement is not among the properties it must establish. Safety and liveness of ordering are inherited; see Section 4.

3 System Architecture

3.1 Overview

Rand is not a blockchain. It is a set of Solana programs plus an off-chain relayer network, and its architecture is best understood by what it delegates rather than what it contains.

Table 1: Separation of Responsibilities

Concern	Provided by	Rand’s role
Ordering, finality	Solana (Proof of History)	none: inherited
Sybil resistance	Solana (Proof of Stake)	none: inherited
Execution	Solana Virtual Machine	deploys programs onto it
Base fees	SOL	pays them via relayers
Asset issuance	Circle, Tether, Solana	none: uses native mints
Shielded state	Rand	commitment tree, nullifier set
Proof verification	Rand	on-chain Groth16 verifier
Sender anonymity	Rand	relayers, in-circuit fees

Concretely, Rand comprises three on-chain programs and one off-chain role:

1. **Shielded Pool Program:** custodies deposited assets, maintains the note commitment tree and nullifier set, and applies validated state transitions
2. **Verifier Program:** checks Groth16 proofs via `alt_bn128`, invoked by the pool program
3. **Relayer Registry:** advertises relayers and their fee expectations, and settles in-circuit SHRUGG fees
4. **Relayers** (off-chain): submit shielded transactions, paying SOL fees so users need not reveal a transparent account

Note that proof *generation* is not an architectural component. A two-input, two-output shielded transfer is provable client-side in a few seconds, so users prove their own transfers; Rand requires no proving network, and we do not posit one.

3.2 Token Model

Rand has one native token. Roles that a standalone Layer 1 would assign to its own staking and gas asset are discharged by SOL, because Rand consumes Solana’s consensus rather than operating its own.

Definition 3.1 (SHRUGG Token). *SHRUGG is the fee asset for confidential transfers. It serves:*

- *Payment for shielded operations, spent from within the pool*
- *Compensation to relayers who submit transactions and pay SOL base fees*
- *Fee asset for confidential transfers of $z\text{USDT}$, the fully-reserved shielded USDT [18]*

Remark 3.2 (On the absence of a staking token). *Earlier versions of this work specified a second token, ATLAS, as staking collateral and gas for public transactions on a Rand Layer 1. Once consensus is inherited from Solana there is no validator set for such a token to secure and no public transaction class for it to price, so it has been removed; SOL occupies that position.*

A related simplification followed. Earlier work proposed USDB, a fractional-algorithmic stablecoin using ATLAS and SHRUGG as volatility absorbers. With ATLAS removed, and with native USDT available on the same chain as the pool, full reserving became the simpler and stronger option: zUSDT [18] is backed one-to-one by native USDT held in a Solana program, and therefore carries no peg mechanism to defend and no reflexive failure mode. SHRUGG’s role is consequently confined to paying for confidentiality, which Section 3.3 shows is a structural requirement.

3.3 Rationale for a Shielded Fee Token

Solana charges base fees in SOL, so it is reasonable to ask why a confidential transfer layer needs a fee token at all. The answer is that transparent fee payment destroys the privacy the layer exists to provide. This is not an economic preference but a structural constraint, and we state it as a theorem.

Theorem 3.3 (Transparent Fee Payment Deanononymises). *If a shielded operation is funded by a transparent fee payment from an account A , then the anonymity set of that operation collapses to the owner of A , irrespective of the strength of the underlying zero-knowledge proof.*

Proof. Let tx_{priv} be a shielded operation whose zero-knowledge proof reveals nothing beyond its public inputs. Solana requires every transaction to be signed by a fee payer and debits that account in SOL. Suppose the user signs as fee payer from account A , where A is linked to public identity id_{pub} through prior transparent activity. The ledger then records

$$\text{tx}_{\text{priv}} \xleftarrow{\text{fee payer signature}} A \xleftarrow{\text{ownership}} \text{id}_{\text{pub}} \quad (3)$$

An observer reads the association directly from the transaction, without attacking the proof system. The proof remains sound and zero-knowledge; the metadata surrounding it is what leaks. Hence anonymity requires that the fee payer be someone other than the user, which in turn requires a mechanism to compensate that party without creating a second transparent link. $\square$

Theorem 3.3 is why Rand employs relayers, and why they must be paid from inside the shielded pool. A relayer signs as fee payer and pays SOL; the user compensates it with SHRUGG deducted within the circuit, so no transparent account belonging to the user ever appears. Paying the relayer transparently, or in SOL from the user’s own account, would reintroduce exactly the linkage the theorem describes.

Proposition 3.4 (Privacy-Preserving Fees). *SHRUGG can be held in shielded form and spent for fees within the zero-knowledge circuit, preserving sender anonymity:*

$$R_{\text{private_gas}} = \left\{ \begin{array}{l} \text{Valid private transfer proof} \\ \wedge \text{ Gas note nullifier valid} \\ \wedge \text{ Gas amount} \geq \text{required fee} \end{array} \right\} \quad (4)$$

3.4 Relay Economics

The relay market is a two-sided auction over a public queue of pending shielded operations. A relay observes an operation carrying an in-circuit fee f_{SHRUGG} , estimates its SOL cost c_{SOL} (base fee, priority fee, and any rent for nullifier accounts), and submits when

$$f_{\text{SHRUGG}} \cdot p_{\text{SHRUGG}} > c_{\text{SOL}} \cdot p_{\text{SOL}} \quad (5)$$

for prevailing prices p . Three properties follow. Relaying is permissionless, since any account with SOL can serve the queue. It is competitive, since relayers are interchangeable and cannot alter the operation they submit: the proof binds all public inputs, so a relayer may only submit or decline. And censorship is weak, since declining an operation leaves it available to every other relayer.

The user’s exposure is bounded. A relayer learns the transaction’s public inputs, which it would learn from the ledger regardless, plus the network origin of the request. The latter is mitigated by submitting over an anonymising transport, and is the residual metadata risk discussed in Section 9.

3.5 Delegated Proving and the Proving Market

The account above assumes the user has already produced the proof, and that the relayer contributes only inclusion. Proving on a laptop takes seconds and is within reach of most users, but not all: constrained clients, mobile wallets, and users transacting frequently may prefer to delegate. Rand therefore admits a second market, in which provers rather than relayers compete, and the two roles collapse into one when a prover also submits.

The mechanism is a race. Pending operations accumulate in a public queue; a prover selects a batch, generates the argument on commodity GPU hardware, signs as fee payer, and submits. The nullifier set adjudicates: the first submission Solana includes consumes the nullifiers and collects the batch’s accumulated f_{SHRUGG} , and every competing submission for the same batch fails verification against the now-published nullifiers and collects nothing while still having paid its own SOL.

Proposition 3.5 (Proving Market Entry). *Entry to the proving market requires neither stake, registration, nor permission. A prover’s expected revenue is $\Pr[\text{win}] \cdot f_{\text{SHRUGG}} \cdot p_{\text{SHRUGG}}$ against a certain cost of proving and submission, so participation is rational when its win probability, determined by its hardware and its latency to Solana’s current leader, exceeds the ratio of cost to fee.*

Two consequences deserve statement rather than assumption. First, a race duplicates work: k provers competing for one batch expend k times the necessary computation, of which $k - 1$ parts are discarded. This is a real efficiency cost, paid to avoid an assignment mechanism, and an assignment mechanism is precisely what would reintroduce the prover-attribution problem — who assigned, under what ordering, and how a verifier knows — that a permissionless race avoids by construction. Second, “first” is not decided by the provers. Inclusion order is Solana’s leader’s to determine, so latency and priority fees influence outcomes alongside proving speed, and a prover with faster hardware but worse connectivity may lose consistently.

Theorem 3.6 (Witness Disclosure Under Delegation). *A prover supplied with the witness w for a transfer learns the sender’s spend authority, the recipient, and the amount, irrespective of the zero-knowledge property of the resulting argument.*

Proof. Zero-knowledge is a property of the argument π with respect to a verifier holding only the statement x . It bounds what π reveals beyond x ; it says nothing about a party holding w directly. The relation R_{transfer} takes as witness the spent notes’ openings, the spend key material authorising them, and the output notes’ values and recipients. A prover evaluating the circuit on w therefore reads exactly these quantities, and the simulator guaranteeing zero-knowledge is irrelevant to a party that never needed to simulate. $\square$

Theorem 3.6 makes the proving market a confidentiality boundary and not merely a performance one, and it is the reason local proving remains the default rather than a legacy path. The honest statement of the tradeoff is that a user delegating a witness exchanges confidentiality against one chosen party for confidentiality against everyone else; this is a weaker guarantee than the pool otherwise provides, and users should choose it deliberately. Blinded delegation — in which the prover receives a re-randomised witness sufficient to satisfy the circuit but insufficient to identify the parties — would remove the disclosure, and is the correct target. We do not claim it here.

3.6 Resource Accounting

A shielded transfer and an ordinary Solana transfer consume different resources, which is what the fee split between SOL and SHRUGG prices.

Table 2: Resource Consumption by Transaction Type

Resource	Transparent Tx	Shielded Tx
SVM computation	Low	Moderate (pairings, tree update)
State storage	Variable	Fixed (commitment, nullifier)
Transaction size	~300 bytes	~800 bytes (proof + inputs)
Client proving	None	2–4 s CPU, ~2 s GPU, or delegated
Verification	Signature only	Groth16 via <code>alt_bn128</code>
Fee asset	SOL	SOL (relayer) + SHRUGG (user)

The SOL component compensates the relayer for costs Solana imposes; the SHRUGG component is what the user can pay without revealing a transparent account. Pricing them in one asset is not possible for the reason given in Theorem 3.3.

3.7 Participant Roles

1. **Users:** deposit into the pool, prove their own transfers client-side by default, and pay SHRUGG fees in-circuit
2. **Relayers:** submit shielded transactions, pay SOL base fees, and collect SHRUGG
3. **Provers:** optionally generate proofs on behalf of users who delegate, competing for batches as described in Section 3.5, and commonly submitting as well
4. **Solana validators:** order and finalise transactions; they run unmodified Agave or Firedancer and need not be aware of Rand

Rand has no validator role and no staking role. The proving role is optional on both sides: users need not delegate, and provers are unbonded and unregistered, so the role adds a market rather than a consensus obligation. This is the intended consequence of building on Solana rather than beside it.

3.8 Solana Virtual Machine Integration

We adopt the Solana Virtual Machine (SVM) [19] as our execution environment.

Table 3: EVM vs SVM Comparison

Feature	EVM	SVM
Execution model	Sequential	Parallel
State model	Unified account	Separated accounts
Typical TPS	15-30 (L1)	5,000-65,000
Smart contract language	Solidity, Vyper	Rust, C, C++
Deterministic fees	No	Yes

Definition 3.7 (SVM Transaction Parallelism). *Two transactions tx_1 and tx_2 can execute in parallel if and only if:*

$$(tx_1.write_accounts \cap tx_2.accounts = \emptyset) \wedge (tx_2.write_accounts \cap tx_1.accounts = \emptyset) \quad (6)$$

4 Settlement on Solana

Rand specifies no consensus protocol. This section records what is inherited, and what that inheritance costs, since the guarantees Rand can offer are bounded by Solana’s.

4.1 Inherited Properties

Solana orders transactions using Proof of History [19], a sequential hash chain acting as a verifiable clock, with stake-weighted leader rotation and a Proof-of-Stake validator set determining canonical history. Rand inherits, without restating or reproofing:

- **Ordering:** a total order over shielded operations, hence a well-defined nullifier insertion sequence
- **Sybil resistance:** economic weight from staked SOL
- **Liveness and censorship resistance:** at Solana’s level, across its validator set
- **Finality:** on Solana’s schedule, not one Rand chooses

4.2 Consequences of Inheritance

Inheritance is not free, and we state the resulting limits plainly.

Rand’s safety is upper-bounded by Solana’s. A consensus failure that reorganised finalised Solana history could reorder nullifier insertions and thereby permit a double-spend within the pool; no property of the shielded construction defends against this, and Rand’s security argument is conditional on Solana’s.

Liveness is likewise inherited. Under congestion, shielded operations compete for block space with all other Solana activity, and relayers must bid priority fees accordingly.

Rand cannot alter the fee asset, the compute budget, or the 1232-byte transaction limit. These are protocol parameters of the host chain and appear throughout this work as fixed constraints rather than design variables; the argument system of Section 2.5 is chosen because of them.

4.3 Concurrency

Solana executes non-conflicting transactions in parallel and admits many transactions per slot, which creates a problem absent from single-writer chains: two shielded transfers proven against

the same commitment tree root may land in the same slot, and the second would be invalid against the root the first produced.

Rand addresses this in two ways. The commitment tree is a concurrent Merkle tree carrying a changelog buffer, permitting many appends per slot without invalidating in-flight proofs. Additionally, the pool program accepts proofs against any of the most recent k roots, retained in a ring buffer:

$$\text{valid}(\pi, \text{rt}) \iff \text{rt} \in \{\text{rt}_h, \text{rt}_{h-1}, \dots, \text{rt}_{h-k+1}\} \quad (7)$$

We take $k = 64$. Enlarging k improves tolerance to proving latency; it also widens the window in which a stale root may be used, which is harmless for soundness (nullifiers are checked against current state regardless) but does weaken anonymity slightly, since the choice of root narrows the set of blocks a transfer could have been constructed against.

5 Privacy-Preserving Transactions

5.1 Private State Model

Private transactions operate on a UTXO-like note model within the commitment tree:

Definition 5.1 (Private Note). *A private note $\text{note} = (v, \text{asset}, \rho, r, pk_{\text{owner}})$ contains:*

- $v \in \mathbb{F}_p$: Value (amount)
- $\text{asset} \in \mathbb{F}_p$: Asset identifier
- $\rho \in \mathbb{F}_p$: Unique note identifier (nullifier seed)
- $r \in \mathbb{F}_p$: Randomness for commitment
- pk_{owner} : Owner's public key

Notes are stored as commitments in a Merkle tree:

$$\text{cm} = H(H(pk_{\text{owner}}) \| v \| \text{asset} \| \rho \| r) \quad (8)$$

Definition 5.2 (Nullifier). *The nullifier for note under secret key sk is:*

$$nf = H(sk \| \rho) \quad (9)$$

Publishing a nullifier marks the note as spent without revealing which note.

5.2 Private Transaction Circuit

A private transaction proves the following relation R_{transfer} :

$$R_{\text{transfer}} = \left\{ \begin{array}{l} ((\text{rt}, \{\text{nf}_i\}, \{\text{cm}_j\}, \text{asset}), \\ (\{\text{note}_i^{\text{in}}\}, \{\text{note}_j^{\text{out}}\}, \{\text{path}_i\}, \text{sk})) : \\ \forall i : \text{MerkleVerify}(\text{rt}, \text{cm}_i^{\text{in}}, \text{path}_i) = 1 \\ \forall i : \text{nf}_i = H(\text{sk} \| \text{note}_i^{\text{in}}. \rho) \\ \forall j : \text{cm}_j = \text{Commit}(\text{note}_j^{\text{out}}) \\ \sum_i \text{note}_i^{\text{in}}.v = \sum_j \text{note}_j^{\text{out}}.v \\ \forall i, j : \text{note}_i^{\text{in}}.\text{asset} = \text{note}_j^{\text{out}}.\text{asset} = \text{asset} \end{array} \right\} \quad (10)$$

5.3 Proof System and On-Chain Verification

Table 4: Groth16 Instantiation

Parameter	Value
Curve	BN254 (alt_bn128)
Argument system	Groth16 [10]
In-circuit hash	Poseidon [11]
Proof size	~256 bytes (3 curve points)
Public inputs	root, 2 nullifiers, 2 commitments, asset, fee
Verification	3 pairings via alt_bn128 syscalls
Setup	per-circuit, discharged by public MPC ceremony

The design target is that proof verification, nullifier insertion, and commitment tree append complete within one transaction, both in size and in compute budget. Size is comfortable: a 256-byte proof plus public inputs and account metadata fits the 1232-byte limit. Compute is the binding constraint, since the pairing check, the tree update, and the program’s own logic share a per-transaction budget.

Remark 5.3 (Figures requiring empirical confirmation). *Published costs for the alt_bn128 syscalls, and the residual budget after a concurrent Merkle tree append, must be measured against the current runtime rather than assumed. Should the combined cost exceed the per-transaction budget, the fallback is to split verification and tree update across two transactions bound by a session account; this preserves the single-proof-upload property, which is the reason Groth16 was chosen, while relaxing atomicity. We flag this explicitly rather than presenting an unverified figure.*

5.4 Nullifier Set

Preventing double-spends requires deciding, for each presented nullifier, whether it has appeared before. Unlike commitments, this demands efficient *non*-membership.

Table 5: Nullifier Set Designs

Design	On-chain cost	Circuit cost	Parallelism
One account per nullifier	rent per spend	none	full
Indexed Merkle tree	root only	non-membership path	limited
Bloom filter	compact	none	full

Rand adopts one program-derived account per nullifier, seeded by the nullifier itself. Existence of the account is non-membership’s negation, decided by the runtime at no circuit cost, and distinct spends touch distinct accounts so they never conflict under parallel execution. The cost is rent per spend, which is bounded and also prices spam. A Bloom filter is unusable here: a false positive would reject a legitimate spend and permanently freeze the note. An indexed Merkle tree removes the rent but requires proving non-membership in-circuit and serialises writers on a single root account, which is the opposite of what Solana’s execution model rewards; it remains the migration path should rent dominate.

5.5 Note Discovery

A recipient must learn of notes addressed to them. This is the least settled part of the design, and we prefer to state that plainly rather than assert a solution.

Table 6: Note Discovery Approaches

Approach	Trust assumption	Cost to client
Trial decryption	none	$O(n)$ in all outputs
Viewing-key indexer	indexer sees all	$O(1)$
Fuzzy message detection	server sees decoy set	$O(pn)$
Oblivious retrieval	none (cryptographic)	$O(1)$, costly server

Rand specifies trial decryption for the initial deployment, with note ciphertexts laid out so that a short prefix suffices to reject a non-matching output. This assumes no additional trust, at the cost of a scan linear in pool activity. We consider a viewing-key indexer unacceptable as a protocol component, since it reconstitutes precisely the observer the system exists to defeat, though nothing prevents a user from delegating to one voluntarily. Fuzzy message detection [12], as deployed by Penumbra, is the intended path for constrained clients: it exposes a tunable false-positive rate rather than the true recipient set. Oblivious message retrieval [13] is the strongest known construction and the eventual goal, currently gated on server-side cost.

5.6 Encrypted Outputs

Recipients learn of incoming notes through encrypted outputs attached to transactions:

$$\text{enc_note} = \text{Enc}_{\text{pk}_{\text{recipient}}}(\text{note}) \quad (11)$$

We use ML-KEM (CRYSTALS-Kyber) [3] for key encapsulation. This is the single most consequential post-quantum decision in the protocol, and the reasoning is asymmetric to that governing proofs. A note ciphertext is published to a permanent public ledger, so an adversary may record it today and decrypt it whenever a sufficiently capable quantum computer exists. No later protocol upgrade can withdraw a ciphertext already published. Confidentiality must therefore be post-quantum from the first transaction, whereas soundness, being a property enforced afresh at each verification by an upgradeable program, need not be. Section 9.3 develops this argument.

6 Supply Auditability and Transparency

6.1 Motivation for Supply Auditability

Because a portion of SHRUGG resides inside the shielded pool and is therefore not directly observable, holders must be able to verify independently that:

1. Total SHRUGG supply is what the emission schedule implies
2. No unauthorised issuance has occurred, whether transparent or shielded
3. Shielded and transparent balances reconcile to the total
4. Burn mechanisms function correctly

Shielding creates a genuine tension: an opaque pool could in principle conceal inflation. The construction below resolves it by making the pool’s aggregate holdings verifiable without revealing any individual note.

6.2 SHRUGG Supply Auditability

Definition 6.1 (SHRUGG Supply Formula). *Since Rand issues no block rewards, supply evolves only through scheduled emission and burns:*

$$S_{SHRUGG}(s) = S_{initial} + \sum_{i=0}^s Emission(i) - \sum_{i=0}^s Burned(i) \quad (12)$$

over Solana slots s , with $S_{initial} = 200,000,000$ SHRUGG.

Emission follows a published schedule evaluated as a function of slot height, so any observer can recompute the expected supply from the mint account alone. Rand deliberately does not tie emission to usage: a demand-responsive schedule would make supply a function of shielded activity, and inferring that activity is exactly the capability the protocol denies observers. Fixing emission avoids constructing a side channel out of monetary policy.

6.3 Privacy-Preserving Supply Verification

Total supply is verifiable from the mint, but reconciling it against the pool requires care, since pool contents are hidden. The pool proves its aggregate solvency without revealing individual notes:

$$R_{\text{supply_audit}} = \left\{ \begin{array}{l} (\text{shielded_balance}, \text{commitment_root}), \\ (\text{all_note_values}, \text{all_randomness}) : \\ \forall \text{notes} : \text{commitment_root contains Commit}(\text{note_value}, \text{randomness}) \\ \wedge \sum(\text{note_values}) = \text{shielded_balance} \\ \wedge \text{All note_values} \geq 0 \end{array} \right\} \quad (13)$$

7 Zero-Knowledge Proof Generation Pipeline

7.1 Overview

Proof generation runs on the user's own machine, not on dedicated hardware. The pipeline is:

$$\text{Rust circuit} \rightarrow \text{R1CS} \rightarrow \text{Witness} \rightarrow \text{MSM over BN254} \rightarrow \text{Groth16 proof} \quad (14)$$

Circuit size, and hence proving time, is dominated by Poseidon invocations: two Merkle paths of depth 32 plus commitment and nullifier derivations. Choosing an algebraic hash rather than SHA-256 is what keeps this in the range where a laptop can prove a transfer in seconds instead of minutes.

7.2 Step 1: Rust Circuit Definition

```
1 pub struct PrivateTransferCircuit {
2     // Public inputs (visible to verifier)
3     pub merkle_root: [u8; 32],
4     pub nullifiers: Vec<[u8; 32]>,
5     pub new_commitments: Vec<[u8; 32]>,
6     pub asset_id: u64,
7
8     // Private inputs (witness, not revealed)
9     notes_in: Vec<Note>,
10    notes_out: Vec<Note>,
```

```

11 merkle_paths: Vec<MerklePath>,
12 secret_key: [u8; 32],
13 }
14
15 impl PrivateTransferCircuit {
16     pub fn build_trace(&self) -> TraceTable {
17         let mut trace = TraceTable::new(TRACE_WIDTH, TRACE_LENGTH);
18
19         // Step 1: Verify Merkle paths for input notes
20         for (i, (note, path)) in self.notes_in.iter()
21             .zip(self.merkle_paths.iter()).enumerate() {
22             let commitment = self.compute_commitment(note);
23             let computed_root = self.verify_merkle_path(commitment, path);
24             trace.set(ROW_START + i * STEPS_PER_NOTE, COL_ROOT, computed_root);
25         }
26
27         // Step 2: Verify value conservation
28         let sum_in: u64 = self.notes_in.iter().map(|n| n.value).sum();
29         let sum_out: u64 = self.notes_out.iter().map(|n| n.value).sum();
30         trace.set(ROW_BALANCE, COL_SUM_IN, sum_in);
31         trace.set(ROW_BALANCE, COL_SUM_OUT, sum_out);
32
33         trace
34     }
35 }

```

Listing 1: Private Transfer Circuit Definition

7.3 Step 2: Rank-1 Constraint System

Definition 7.1 (R1CS). *An R1CS instance is a set of triples (a_i, b_i, c_i) of vectors over $\mathbb{F}_p$; a witness w satisfies it when*

$$\forall i : \langle a_i, w \rangle \cdot \langle b_i, w \rangle = \langle c_i, w \rangle \quad (15)$$

Groth16 proves satisfiability of a fixed R1CS instance, which suits Rand precisely because the shielded transfer circuit is fixed. The per-circuit trusted setup that Groth16 requires is therefore paid once.

Remark 7.2 (General SVM execution is out of scope). *An earlier formulation of this work claimed that arbitrary SVM instructions could be expressed as constraint systems, enabling zero-knowledge execution of general Solana programs. That claim is withdrawn here. While true in principle, a zkVM for sBPF is a substantially larger undertaking than a fixed transfer circuit, and its proof sizes and proving costs are not those reported below. Rand proves one relation: a shielded transfer. General verifiable execution on Solana is left to future work and should not be inferred from this specification.*

7.4 Performance Characteristics

Table 7: ZK Proving Performance by Operation Type

Operation Type	Complexity	Proof Overhead	Example
Field arithmetic (+, -, ×)	Fast (1-10 cycles)	Low	Balance calculations
Hash functions (Poseidon)	Medium (100-500)	Medium	Commitment generation
Hash functions (SHA-256)	Very Slow (10K+)	Very High	Legacy hash verification
Signature verification	Slow (5K+ cycles)	High	EdDSA verification
Range checks	Medium (50-200)	Medium	Value bounds checking
Division/Modulo	Slow (500+ cycles)	High	Requires extended GCD
Bitwise operations	Slow (10-50/bit)	High	Binary decomposition

7.5 End-to-End Proving Timeline

For a two-input, two-output shielded transfer, proven client-side:

Table 8: Client-Side Proving, Commodity CPU

Stage	Time
Witness generation	60 ms
Poseidon Merkle path constraints (2×32)	400 ms
Constraint solving	300 ms
Multi-scalar multiplication (BN254)	1,600 ms
Serialisation	5 ms
Total proving time	~2.4 seconds
Proof size	256 bytes
On-chain verification	3 pairings

Remark 7.3. *These are design targets derived from published Groth16 benchmarks for circuits of comparable constraint count, not measurements of an implementation. They are stated to establish feasibility of client-side proving, which is the architecturally load-bearing claim: at seconds on a laptop, no proving network is required, and the protocol therefore needs no reward mechanism for provers. Should measurement place proving in the tens of seconds, mobile clients would need to delegate proving, which reintroduces a metadata leak to the delegate and would require revisiting Section 5.5.*

8 Token Economics

8.1 Initial Token Distribution

Table 9: Initial SHRUGG Distribution

Allocation	SHRUGG (%)
Core Development Team	15
Foundation/Treasury	20
Ecosystem Development	20
Early Relayers	10
Private Sale	5
Public Sale	10
Community Airdrops	5
Liquidity Reserve	15
Total Initial Supply	200M SHRUGG

There is no staking rewards reserve, because Rand has nothing to stake: security is purchased from Solana’s validator set, paid for in SOL fees at the moment of use rather than through inflation. This is a structural economic consequence of building on Solana, and it means SHRUGG’s value derives from demand for confidential transfers rather than from a yield.

8.2 Emission

Definition 8.1 (Emission Schedule). *Emission decays geometrically toward a floor, evaluated on Solana slot height:*

$$\iota(y) = \max\{\iota_\infty, \iota_0 \cdot (1 - \delta)^y\} \quad (16)$$

with $\iota_0 = 6\%$, $\delta = 15\%$ per year, and $\iota_\infty = 1\%$.

As noted in Section 6, emission is deliberately independent of shielded activity. A usage-responsive schedule would leak aggregate private volume through the supply curve.

8.3 Fee Distribution

Each shielded operation carries an in-circuit SHRUGG fee, split as follows.

Table 10: Shielded Operation Fee Distribution

Component	Share	Rationale
Relayer	70%	must cover SOL base and priority fees
Treasury	20%	protocol development, ceremony upkeep
Burn	10%	ties supply to usage

The relayer share floats with SOL’s price and with network congestion: relayers decline operations whose fee fails to cover their costs, so the effective floor is set by the market rather than by protocol constant. Users observe prevailing relayer fees before constructing a transfer.

9 Security Analysis

9.1 Inherited Consensus Security

Rand proves no consensus properties of its own. Safety and liveness of ordering are Solana's, and Rand's guarantees are conditional on them.

Proposition 9.1 (Conditional Safety). *Assume Solana's consensus provides finality: a finalised transaction is never reordered or reverted. Then no note in the Rand pool can be spent twice.*

Proof. Each note yields a unique nullifier $\text{nf} = \text{Poseidon}(\text{nk}||\rho)$, and the circuit enforces its correct derivation. On the first spend, the pool program creates the account seeded by nf ; account creation fails if the account exists. A second spend of the same note produces the same nullifier and therefore attempts to create an existing account, which the runtime rejects. Both spends are ordered by assumption, so exactly one precedes the other and exactly one succeeds. $\square$

Remark 9.2 (Where the assumption binds). *The proof depends entirely on finality. Were finalised Solana history reorganised, nullifier creations could be reordered or dropped and a double-spend would become possible. Rand cannot defend against this, and we do not claim otherwise: its security is upper-bounded by that of the chain it is deployed on. This is the price of not running a validator set, and it is the central tradeoff of the architecture.*

9.2 Privacy Security

Theorem 9.3 (Transaction Privacy). *Under the collision resistance of Poseidon and the zero-knowledge property of Groth16, a shielded transfer reveals nothing beyond its public inputs: the tree root, the nullifiers, the output commitments, the asset identifier, and the fee.*

Remark 9.4 (What the theorem does not cover). *Cryptographic privacy of the transfer does not imply unobservability of the user. Four leaks lie outside the proof system and are not addressed by it. The asset identifier and fee are public, so transfers are distinguishable by asset and, to a degree, by fee. A relay learns the network origin of a submission, which requires an anonymising transport to mitigate. The anonymity set is only as large as the pool is active: a transfer within a pool of few participants is formally private and practically identifiable. And a user who delegates proving discloses the witness in full to the prover, by Theorem 3.6, which is a disclosure to a chosen counterparty rather than to the public but is a disclosure nonetheless. Section 5.5 records a further leak in the discovery mechanism. We enumerate these because a privacy system whose stated guarantees exceed its real ones is worse than one with modest, accurate claims.*

Theorem 9.5 (Double-Spend Prevention). *A valid transaction cannot spend the same note twice.*

Proof. See the proof of Conditional Safety above; the mechanism is nullifier account creation, and the argument is identical. $\square$

9.3 Post-Quantum Security

Rand's post-quantum posture is deliberately asymmetric, and since this departs from the usual "post-quantum throughout" claim we set out the reasoning rather than the conclusion alone.

The distinction is between properties that are permanently exposed once published and properties re-established at each use.

Table 11: Post-Quantum Posture by Property

Property	Primitive	PQ	Exposure
Note confidentiality	ML-KEM	yes	permanent, on-ledger
Commitment hiding	Poseidon	yes	permanent, on-ledger
Spend authorisation	ML-DSA	yes	permanent, on-ledger
Proof soundness	Groth16/BN254	no	live, per-verification

Proposition 9.6 (Asymmetry of Quantum Exposure). *Confidentiality of published ciphertexts must be post-quantum from first use, whereas soundness of an upgradeable verifier need not be.*

Proof. A ciphertext published at time t_0 is recorded permanently by any observer. If its security rests on an assumption broken at some later t_1 , the plaintext is recoverable at t_1 ; no action taken during (t_0, t_1) can prevent this, since the ciphertext cannot be unpublished. Confidentiality is therefore irrevocable and must anticipate the strongest future adversary. Soundness is different: it is enforced by a verifier executed afresh at each verification. Replacing that verifier before t_1 preserves soundness for all subsequent proofs, and proofs verified before t_1 were checked under an assumption then unbroken. Since Solana programs are upgradeable, the verifier can be replaced. $\square$

We therefore accept a classically-sound argument system in exchange for single-transaction verification, and reject any compromise on confidentiality. The concrete migration trigger is the availability of a succinct post-quantum argument whose proofs fit a Solana transaction and whose verification fits the compute budget; lattice-based and hash-based succinct arguments are both plausible candidates, and neither is practical at the time of writing.

Remark 9.7 (Honest statement of the limitation). *Rand should not be described as an end-to-end post-quantum system. An adversary with a sufficiently capable quantum computer, acting before the verifier is upgraded, could forge a proof and mint value from the pool. Such an adversary could not, however, decrypt historical notes or deanonymise past transfers. This is a considered position, not an oversight, and we prefer to state it than to let the stronger claim stand unqualified.*

10 Implementation Considerations

10.1 Development in Rust

All core components are implemented in Rust, leveraging:

- Memory safety without garbage collection
- Zero-cost abstractions
- Fearless concurrency
- Strong type system

10.2 Performance Targets

Table 12: System Performance Targets

Metric	Target
Client proving time	<3 seconds, commodity CPU
Proof size	256 bytes
Transactions per shielded transfer	1
Settlement latency	Solana finality
Commitment tree depth	32 (4.3×10^9 notes)
Recent-root window	64
Supported assets	native USDC, USDT, SOL

Throughput is not a Rand parameter. Shielded transfers occupy ordinary Solana transactions and compete for block space on equal terms, so the ceiling is Solana’s and varies with network conditions.

11 Related Work

Privacy-Preserving Layer 1s. Zcash [6] established the note, commitment, and nullifier model on which Rand’s circuit is based, and Monero [8] achieves comparable goals with ring signatures and stealth addresses. Both are independent networks with their own assets and security budgets. Rand adopts Zcash’s cryptographic structure while rejecting its deployment model: the anonymity is useful, but requiring users to leave Solana and its native stablecoins is precisely the cost we are unwilling to pay.

Confidential Transfers on Solana. Token-2022’s confidential transfer extension [15] encrypts balances using twisted ElGamal with proofs verified by a native program. It hides amounts but not participants, and cannot be enabled on the already-deployed native USDC and USDT mints. It is complementary rather than competing: where amount confidentiality suffices and the mint is under one’s control, it is the simpler choice.

Zero-Knowledge Infrastructure on Solana. Light Protocol [16] demonstrated Groth16 verification on Solana via the `alt_bn128` syscalls, establishing the feasibility on which Rand’s verification design rests. Its ZK compression targets state cost rather than privacy, but the verification path is shared.

Confidential Computation Networks. Arcium [17] provides confidential computation for Solana through multi-party computation. The trust assumption differs fundamentally: MPC requires a threshold of non-colluding nodes, whereas Rand’s confidentiality is cryptographic and requires trust in no operator. MPC in exchange supports richer computation than a fixed transfer circuit.

Note Discovery. Fuzzy message detection [12] and oblivious message retrieval [13] address the problem of Section 5.5; Penumbra deploys the former. Rand’s initial reliance on trial decryption is a known weakness rather than a contribution.

Proof-of-Useful-Work. Earlier versions of this work positioned Rand as a proof-of-useful-work consensus protocol, in the lineage of Primecoin [7]. That framing has been abandoned. Once consensus is inherited from Solana there is no work to reward, and the observation that proof generation is cheap enough to perform client-side removes the remaining justification for a proving market.

12 Conclusion

We have presented Rand, a privacy layer for Solana rather than a blockchain. The central claim is architectural: transaction privacy does not require a new chain, and building one imposes costs, in bridged assets, fragmented liquidity, and a separate security budget, that a program deployed onto an existing chain avoids entirely. Rand’s shielded pool operates over the native USDC and USDT already circulating on Solana, and inherits its ordering, finality, and throughput.

Two results structure the design. Transparent fee payment deanonymises a shielded operation regardless of proof strength, which makes an in-circuit fee token and a relay market structural requirements rather than economic choices; this is what SHRUGG is for. And quantum exposure is asymmetric between confidentiality and soundness, which justifies post-quantum note encryption alongside a classically-sound, upgradeable verifier. Both conclusions were reached by following platform constraints, particularly Solana’s 1232-byte transaction limit, rather than by preference.

We are explicit about what remains unresolved. Note discovery has no satisfying solution, and trial decryption will not serve constrained clients. The compute budget for combined verification and tree update requires measurement against the live runtime. The anonymity set must reach meaningful size before formal privacy becomes practical privacy. And Rand’s safety is bounded above by Solana’s, which is the considered price of not operating a validator set.

Future work includes the trusted setup ceremony, formal verification of the circuit, migration to fuzzy message detection, and monitoring for a succinct post-quantum argument system whose verification fits a Solana transaction, which would close the last gap in the post-quantum posture.

Acknowledgments

We thank the broader cryptocurrency research community for foundational work on consensus mechanisms, zero-knowledge proofs, and privacy-preserving systems.

References

- [1] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity,” in *CRYPTO*, 2018.
- [2] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Keccak,” in *EUROCRYPT*, 2013.
- [3] J. Bos et al., “CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM,” in *Euro S&P*, 2018.
- [4] L. Ducas et al., “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *TCHES*, 2018.
- [5] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *STOC*, 1996.
- [6] D. Hopwood et al., “Zcash protocol specification,” 2016.
- [7] S. King, “Primecoin: Cryptocurrency with prime number proof-of-work,” 2013.
- [8] S. Noether, “Ring signature confidential transactions for Monero,” *IACR ePrint*, 2015.
- [9] J. O’Connor et al., “BLAKE3: One function, fast everywhere,” 2020.
- [10] J. Groth, “On the size of pairing-based non-interactive arguments,” in *EUROCRYPT*, 2016.

- [11] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A new hash function for zero-knowledge proof systems,” in *USENIX Security*, 2021.
- [12] G. Beck, J. Len, I. Miers, and M. Green, “Fuzzy message detection,” in *ACM CCS*, 2021.
- [13] Z. Liu and E. Tromer, “Oblivious message retrieval,” in *CRYPTO*, 2022.
- [14] Solana Foundation, “`alt_bn128` syscalls,” Solana documentation.
- [15] Solana Foundation, “Token-2022 confidential transfer extension,” Solana Program Library documentation.
- [16] Light Protocol, “ZK compression on Solana,” technical documentation.
- [17] Arcium, “Confidential computing network for Solana,” technical documentation.
- [18] D. Suhubdy, “zUSDT: A fully-reserved shielded representation of USDT on Solana,” Bitwyre Research, 2025.
- [19] A. Yakovenko, “Solana: A new architecture for a high performance blockchain,” 2018.